example, be executed on one CPU using time slicing, where every process gets a certain amount of time to run on the CPU, one after another.

The JVM has a standard concurrent programming model (see figure 1.14) where, roughly speaking, processes are expressed in objects and methods, which are executed on threads. Threads can be executed on many CPUs in parallel or using a sharing mechanism like time slicing on one CPU. As discussed earlier, threads can't be applied directly to scaling out, only to scaling up.

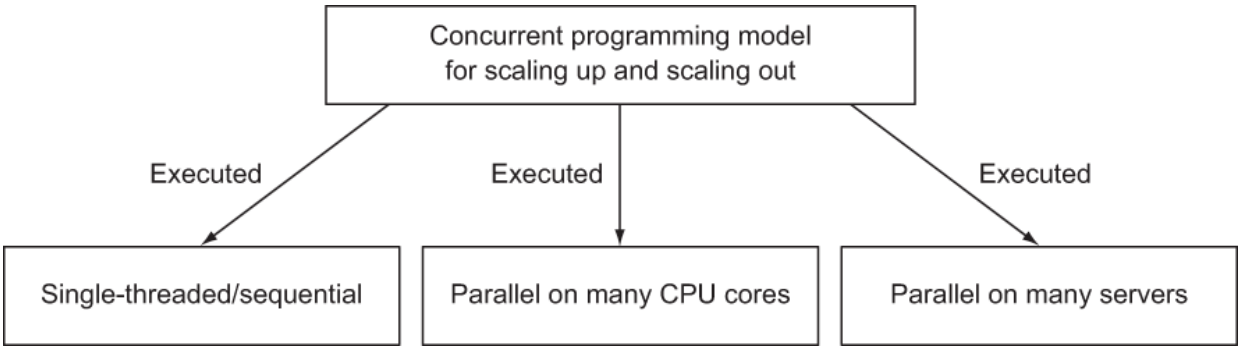


Figure 1.14 Concurrent programming model

The concurrent programming model that you're after should function for one CPU or many and for one server or many servers. The actor model chooses the abstraction of sending and receiving messages to decouple from the number of threads or servers being used.

1.6.1 An asynchronous model

If you want the application to scale to many servers, there's an important requirement for the programming model: it must be *asynchronous*, allowing components to continue working while others haven't responded yet, as in the chat application.